

Java 9 → Java 21 Hands-On Lab Assignments

General Lab Setup

Recommended JDK: Java 21 LTS

For features that were preview features in earlier releases, use Java 21 to practice their finalized forms unless the assignment specifically asks you to explore the historical preview.

Suggested structure:

```
java-modern-features-lab/
├── lab01-jshell/
├── lab02-interface-private-methods/
├── lab03-collection-factories/
├── lab04-modules/
├── lab05-var/
├── lab06-http-client/
├── lab07-string-enhancements/
├── lab08-files/
├── lab09-switch-expression/
├── lab10-text-blocks/
├── lab11-pattern-instanceof/
├── lab12-records/
├── lab13-sealed-classes/
├── lab14-pattern-switch/
├── lab15-record-patterns/
├── lab16-record-constructors/
├── lab17-advanced-sealed/
└── lab18-string-templates/
```

Note: String Templates were a preview feature and were **withdrawn in JDK 23**. Therefore, they are useful as a historical Java 21 preview exercise, but they should not be taught as a current Java feature.

1. Java 9

Lab 1 — JShell as a Java Development Tool

Objective

Learn to use JShell for rapid experimentation without creating .java files.

Assignment

Use JShell to build a small employee-processing experiment.

Perform the following operations:

1. Create an employee name.
2. Create an employee salary.
3. Calculate annual salary from monthly salary.
4. Create a list of employee salaries.
5. Calculate:
 - maximum salary
 - minimum salary
 - average salary
6. Create a method:

`calculateBonus(double salary)`

7. Create a method that determines whether an employee is eligible for a bonus.
8. Display the variables and methods currently defined in JShell.
9. Modify one of the variables.
10. Save the JShell session to a file.

Hints

- Start with `jshell`.
- Use `/vars` to inspect variables.
- Use `/methods` to inspect methods.
- Use `/list` to inspect entered code.
- JShell supports method declarations directly.
- Explore `/save` and `/open`.

Challenge

Create a small stream pipeline directly inside JShell:

```
List<Integer> salaries
    ↓
filter
    ↓
map
    ↓
average
```

2. Private Methods in Interfaces

Objective

Understand why Java 9 introduced private interface methods.

Assignment

Create an interface:

PaymentProcessor

with these public default methods:

```
processCreditCard()
processDebitCard()
processUPI()
```

All three methods perform some common validation.

Refactor the common validation logic into a **private interface method**.

Expected Design

Conceptually:

```
PaymentProcessor
|
+-- processCreditCard()
|
+-- processDebitCard()
|
+-- processUPI()
    |
    +---- private validate()
```

Hints

- A private interface method can be called by methods inside the same interface.
- It cannot be called by implementing classes.
- Use default methods for behavior that implementing classes can inherit.

Challenge

Add another default method that performs common logging using another private method.

3. Collection Factory Methods

Objective

Practice Java 9 immutable collection factory methods.

Assignment

Create collections representing:

Departments

Roles

Employee IDs

Use factory methods instead of manually constructing ArrayList, HashSet, and HashMap.

Create:

List<String>

Set<String>

Map<Integer, String>

using Java 9 factory methods.

Then attempt:

1. Add an element.
2. Remove an element.
3. Modify a map value.

Observe what happens.

Hints

Look at:

```
List.of(...)  
Set.of(...)  
Map.of(...)  
Map.ofEntries(...)
```

Challenge

Create a configuration map:

```
application.name  
application.version  
application.environment
```

and ensure that nobody can modify it after creation.

Discussion

Why are these factory methods useful for configuration data?

4. JPMS — Java Module System

Assignment 4A — Create Your First Module

Create two modules:

```
employee.model  
employee.service
```

employee.model

Create:

```
Employee  
Department
```

employee.service

Create:

```
EmployeeService  
Main
```

The service module should use Employee.

Requirements

Create:

`module-info.java`

for both modules.

The service module should:

`requires employee.model;`

The model module should export the appropriate package.

Hints

The module descriptor contains declarations such as:

`module ...`

and:

`exports ...`

and:

`requires ...`

5. JPMS — Strong Encapsulation

Assignment

Create:

`employee.model`

with two packages:

`com.example.employee.api`

`com.example.employee.internal`

Put:

`Employee`

in the API package.

Put:

`EmployeeValidator`

in the internal package.

Export only:

`com.example.employee.api`

Task

From another module:

1. Access `Employee`.
2. Try to access `EmployeeValidator`.

Observe the compilation failure.

Learning Goal

Understand that:

`public class`

does **not** automatically mean:

`public` outside the module

The package must also be exported.

Challenge

Explain the difference between:

`public`

and:

`exports`

in JPMS.

Java 10 & 11

6. var — Local Variable Type Inference

Assignment

Rewrite the following code using `var` wherever appropriate:

```
List<String> employees = new ArrayList<>();
Map<Integer, Employee> employeeMap = new HashMap<>();
Employee employee = new Employee();
String name = employee.getName();
```

Then identify situations where var **cannot** be used.

Examples to investigate:

method parameters
return types
class fields
null initialization

Challenge

Determine the inferred type of each:

```
var x = 10;  
var y = 10.0;  
var z = "Java";  
var list = List.of("A", "B");
```

Discussion

Why is this called **type inference** rather than dynamic typing?

7. Java 11 HTTP Client API

Assignment — REST Client

Build a Java application that calls a public REST API.

The application should:

1. Create an `HttpClient`.
2. Send a GET request.
3. Display the HTTP status.
4. Display the response body.
5. Send a POST request with JSON.
6. Add request headers.
7. Handle errors.

Hints

Investigate:

`HttpClient`
`HttpRequest`
`HttpResponse`

and:

`HttpClient.newHttpClient()`

For asynchronous processing investigate:

`sendAsync()`

Challenge

Make two HTTP requests asynchronously and combine their results using `CompletableFuture`.

8. String Enhancements

Assignment

Create a program that processes user-entered text.

Use the newer String APIs to:

1. Check whether a string is blank.
2. Remove leading/trailing whitespace.
3. Strip only leading whitespace.
4. Strip only trailing whitespace.
5. Repeat a string.
6. Split a multiline string into lines.

Hints

Investigate:

`isBlank()`
`strip()`
`stripLeading()`
`stripTrailing()`
`repeat()`
`lines()`

Challenge

Given:

" Java\nSpring Boot\nKafka\nKubernetes "

produce a clean list of technologies.

9. Files API Improvements

Assignment — Log Analyzer

Create a text file:

`application.log`

containing lines such as:

```
INFO Application started
INFO User logged in
ERROR Database connection failed
INFO Request completed
ERROR Payment failed
```

Write a program that:

1. Reads the file.
2. Counts ERROR messages.
3. Displays all ERROR lines.
4. Creates a new file containing only ERROR messages.
5. Appends a summary to another file.

Hints

Investigate:

```
Files.readString()
Files.writeString()
Files.lines()
```

Challenge

Process the file using a Stream without explicitly storing all lines in memory.

Java 12–15

10. Switch Expressions

Assignment — Order Pricing

Create:

```
enum CustomerType {
    REGULAR,
    PREMIUM,
```

```
    VIP  
  }
```

Calculate discount using a switch expression.

Example business rules:

```
REGULAR → 5%  
PREMIUM → 10%  
VIP     → 20%
```

Requirements

Return the discount directly from the switch.

Hints

Investigate:

```
case ... ->
```

and:

```
yield
```

Challenge

Create a second switch expression that calculates shipping charges based on:

```
STANDARD  
EXPRESS  
OVERNIGHT
```

11. Text Blocks

Assignment — JSON Payload

Create a JSON request body representing:

```
Customer  
├── id  
├── name  
├── email  
└── address
```

First write it using traditional string concatenation.

Then rewrite it using a text block.

Compare

Traditional:

```
"{" +  
  "\"id\": 101," +  
  ...
```

versus a multiline text block.

Challenge

Create an SQL query using a text block:

```
SELECT ...  
FROM ...  
WHERE ...
```

Discuss why text blocks improve readability but **do not themselves make SQL safe from injection**.

12. Pattern Matching for instanceof

Assignment — Shape Processor

Create:

```
Shape  
Circle  
Rectangle  
Triangle
```

Given a Shape, calculate its area.

Use pattern matching instead of:

`instanceof`

followed by an explicit cast.

Hints

Investigate the form:

```
if (obj instanceof SomeType variable)
```

Challenge

Add multiple conditions to the pattern:

```
instanceof  
+  
boolean condition
```

and explain why the variable is only available where the pattern definitely matched.

Java 16–17

13. Records

Assignment — Employee DTO

Replace a traditional immutable DTO:

EmployeeDTO

with a record containing:

```
id  
name  
department  
salary
```

Test

Demonstrate:

1. Constructor generation.
2. Accessor methods.
3. equals().
4. hashCode().
5. toString().

Challenge

Use the record as a key in a HashMap.

Explain why correct equals() and hashCode() behavior is important.

14. Record Validation

Assignment

Create:

```
Employee(int id, String name, double salary)
```

as a record.

Business rules:

```
id > 0
```

```
name must not be blank
```

```
salary >= 0
```

Reject invalid employees.

Hint

Use a **compact canonical constructor**.

Conceptually:

```
public Employee {  
    validation...  
}
```

Challenge

Normalize the name inside the constructor.

For example:

```
" john smith "
```

should become:

```
"John Smith"
```

Then discuss an important point:

Records provide shallow immutability, not deep immutability.

15. Sealed Classes

Assignment — Payment Hierarchy

Create:

```
sealed interface Payment
```

with permitted implementations:

`CreditCardPayment`

`UPIPayment`

`BankTransferPayment`

Prevent other classes from implementing `Payment`.

Requirements

Create a processor:

`PaymentProcessor`

that processes each payment type.

Hints

Investigate:

`sealed`

`permits`

`final`

`non-sealed`

Challenge

Make one implementation:

`non-sealed`

and demonstrate how that changes the inheritance rules.

16. Strong Encapsulation of JDK Internals

Assignment — Legacy Code Investigation

Find an example of code that uses a JDK internal API such as classes under:

`sun.*`

`com.sun.*`

Investigate:

1. Why the API is considered internal.
2. Whether it works on modern JDKs.
3. What public replacement API should be used.

Deliverable

Create a short table:

Internal API	Purpose	Public Replacement
Internal API	What it did	Recommended API

Discussion

Why does Java want applications to avoid depending on JDK internals?

17. macOS Rendering Pipeline

Investigation Assignment

This feature is primarily relevant to applications using Java's desktop graphics stack on macOS.

Research and demonstrate, where possible:

Java 17
↓
macOS
↓
Metal rendering pipeline

Task

Create a simple Java Swing application containing:

- JFrame
- JPanel
- custom drawing
- animation

Run it on macOS with Java 17+.

Investigation

Determine:

1. Which rendering pipeline is being used.
2. How Java 17 changed macOS rendering.
3. Why this change is largely invisible to ordinary server-side Java applications.

For enterprise Java training, treat this as an **investigation/demo assignment**, rather than a core programming lab.

Java 18 → Java 21

18. Pattern Matching for switch

Assignment — Payment Processor

Create:

```
Payment
CreditCard
UPI
BankTransfer
Cash
```

Process the payment using a switch expression.

Instead of:

```
if
else if
else if
```

use pattern matching with switch.

Example design

```
Payment
|
+-- CreditCard
+-- UPI
+-- BankTransfer
```

Hints

Investigate:

```
case CreditCard c ->
```

and:

```
default ->
```

Challenge

Make the compiler enforce exhaustive handling using a sealed hierarchy.

19. Guarded Patterns

Assignment — Employee Classification

Create:

Employee

with:

name

salary

Classify employees:

salary $\geq$ 1,000,000 → Executive

salary $\geq$ 500,000 → Senior

otherwise → Regular

Use pattern matching plus conditions.

Hint

In Java's evolution of pattern matching, early previews used guarded forms such as when/guard concepts differently across releases. For Java 21, practice the finalized conditional switch style rather than teaching an older preview syntax as if it were current.

Challenge

Add:

VIP employee

when:

salary $\geq$ 1,000,000

AND

employee has more than 10 years experience

20. Record Patterns

Assignment — Customer Address

Create nested records:

```
record Address(  
    String city,  
    String state  
) {}
```

```
record Customer(  
    int id,  
    String name,  
    Address address  
) {}
```

Given a Customer, extract:

```
name  
city  
state
```

using record patterns.

Objective

Compare:

Traditional

```
Customer customer
```

```
customer.name()  
customer.address().city()  
customer.address().state()
```

with pattern-based decomposition.

Challenge

Use nested record patterns.

21. Pattern Matching + Records + Sealed Classes

This is the **major integration lab**.

Assignment — Insurance Claim Processor

Design:

sealed interface Claim

with:

MedicalClaim

VehicleClaim

TravelClaim

Represent each claim using records.

Example:

MedicalClaim

claimId

amount

patient

VehicleClaim

claimId

amount

vehicle

Use:

sealed classes

+

records

+

record patterns

+

switch patterns

to calculate the approved claim amount.

Business Rules

For example:

MedicalClaim

amount <= 100000 → approve 100%

VehicleClaim

amount <= 500000 → approve 80%

TravelClaim

amount <= 200000 → approve 90%

Challenge

Make the compiler help you identify when a new claim type is added but the processor has not been updated.

This assignment demonstrates the **real design benefit** of these features.

22. Canonical Constructors

Assignment — Product Record

Create:

```
record Product(  
    int id,  
    String name,  
    double price  
)
```

Implement validation using the canonical constructor.

Rules:

```
id > 0  
name not blank  
price >= 0
```

Advanced Requirement

Normalize:

name

and round the price according to your chosen business rule.

Questions

Learners should answer:

1. What is the canonical constructor?
 2. What is a compact constructor?
 3. What happens if you don't define one?
 4. Why are record fields final?
 5. Can a record have mutable fields?
-

23. Deep vs Shallow Immutability

Assignment

Create:

```
record Employee(  
    int id,  
    String name,  
    List<String> skills  
)
```

Create an employee and modify the original `skills` list.

Observe whether the record's state changes.

Task

Modify the design to achieve stronger immutability.

Hint

The important question is:

Is the reference immutable, or is the object referenced by that reference immutable?

Investigate:

`List.copyOf()`

Challenge

Demonstrate the difference between:

`List.copyOf()`

and simply assigning the incoming list.

24. Advanced Sealed Classes

Assignment — Banking Domain

Design:

sealed interface Account

with:

SavingsAccount
CurrentAccount
InvestmentAccount

Allow only selected extension points.

Experiment with:

final
sealed
non-sealed
permits

Requirements

Create an inheritance tree where:

```
Account
|
+-- SavingsAccount
|
+-- CurrentAccount
    |
    +-- CorporateCurrentAccount
```

Control exactly where extension is allowed.

Discussion

Ask learners:

- Where should inheritance be closed?
 - Where should it remain open?
 - Why might a library designer use sealed types?
 - How does this help pattern matching?
-

25. String Templates — Historical Java 21 Preview

Assignment

Create a customer message:

Customer <name> has an outstanding balance of <amount>.

Experiment with Java 21's String Templates preview feature.

Investigate:

STR

and template expressions.

Security Exercise

Consider:

```
SELECT * FROM users WHERE name = '<user input>'
```

Discuss why simply inserting user input into a string is dangerous.

Then compare template interpolation with **parameterized SQL**.

Critical Learning Point

Do **not** conclude:

“String Templates automatically prevent SQL injection.”

Instead:

String templates provide structured interpolation mechanisms, but SQL injection prevention comes from using appropriate database APIs such as prepared/parameterized statements.

Research Task

Investigate why String Templates were subsequently withdrawn from the Java language roadmap.

26. Capstone Lab — Modern Java Enterprise Order System

This assignment combines nearly everything.

Scenario

Build a small order-processing application.

Domain

Customer
Product
Order

OrderItem
Payment

Requirements

Use:

Java 9

- Collection factory methods
- Private interface methods
- JPMS

Java 10

- var

Java 11

- HTTP Client
- String enhancements
- Files API

Java 14+

- Switch expressions
- Pattern matching

Java 15+

- Text blocks

Java 16+

- Records
- Pattern matching for instanceof

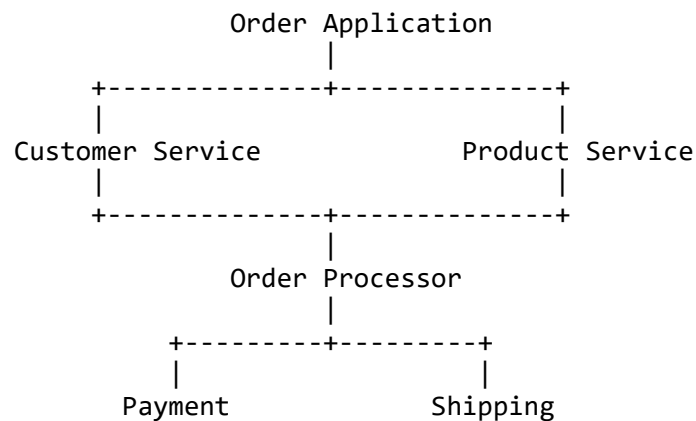
Java 17

- Sealed classes

Java 21

- Pattern matching for switch
 - Record patterns
 - Advanced sealed hierarchy
-

Suggested Architecture



Functional Requirements

The application should:

1. Load products from a file.
 2. Call an HTTP service to retrieve customer information.
 3. Create an order.
 4. Validate the order.
 5. Calculate discounts.
 6. Calculate shipping.
 7. Process payment.
 8. Generate an invoice.
 9. Write the invoice to a file.
 10. Log processing results.
-

Final Challenge — Feature Identification

Give learners the following code/design problems and ask:

Which modern Java feature would you choose and why?

Problem 1

You need a small immutable data carrier.

Expected: Record

Problem 2

You have a hierarchy that should allow only three implementations.

Expected: Sealed class/interface

Problem 3

You need different behavior based on the runtime type of an object.

Expected: Pattern matching for switch

Problem 4

You want to extract components from nested immutable data structures.

Expected: Record patterns

Problem 5

You need a read-only configuration collection.

Expected: `List.of()`, `Set.of()`, `Map.of()`

Problem 6

You want to experiment with Java without creating source files.

Expected: JShell

Problem 7

You need a lightweight DTO with validation.

Expected: Record + canonical/compact constructor

Problem 8

You want to expose only selected packages from a library.

Expected: JPMS + exports

Problem 9

You need to make an HTTP call without adding a third-party HTTP client.

Expected: Java HTTP Client API

Problem 10

You need multiline JSON/SQL/HTML in source code.

Expected: Text blocks
